

BUILDING A PORTABLE AGENT HARNESS

*A textbook and practical workbook for understanding,
designing, implementing, and verifying agentic systems*

@kwis7
github.com/kwis7

Version 2.1 textbook draft | 9 August 2026

Contents

CORE CHAPTERS

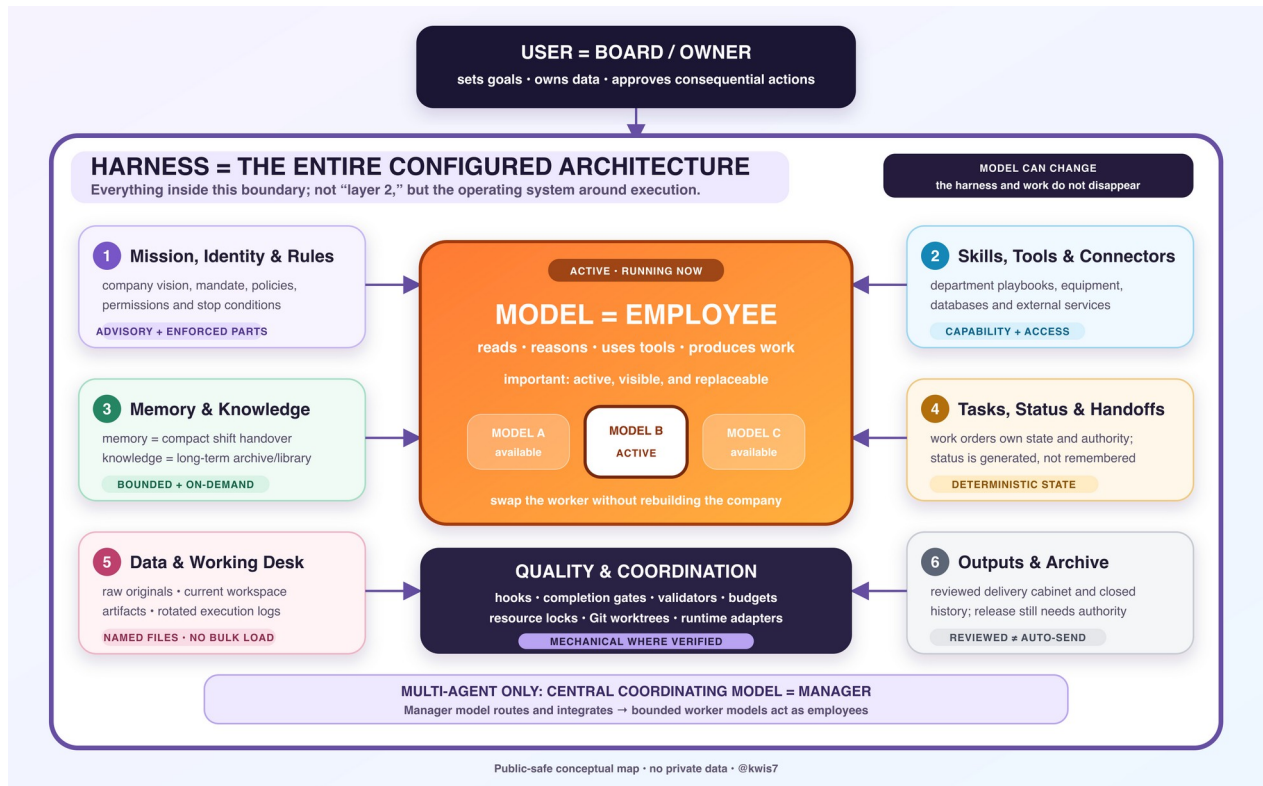
Part 1: Foundations — from a model to an agentic system	7
Part 2: Context, state, memory, knowledge, and files	12
Part 3: Architecture and personalisation	19
Part 4: Reliability, governance, and enforcement	25
Part 5: Build workbook — from interview to verified harness	32

REFERENCE APPENDICES

Appendix A: Runtime adapter matrix	42
Appendix B: Governance schemas and command reference	44
Appendix C: The short starter prompt	47

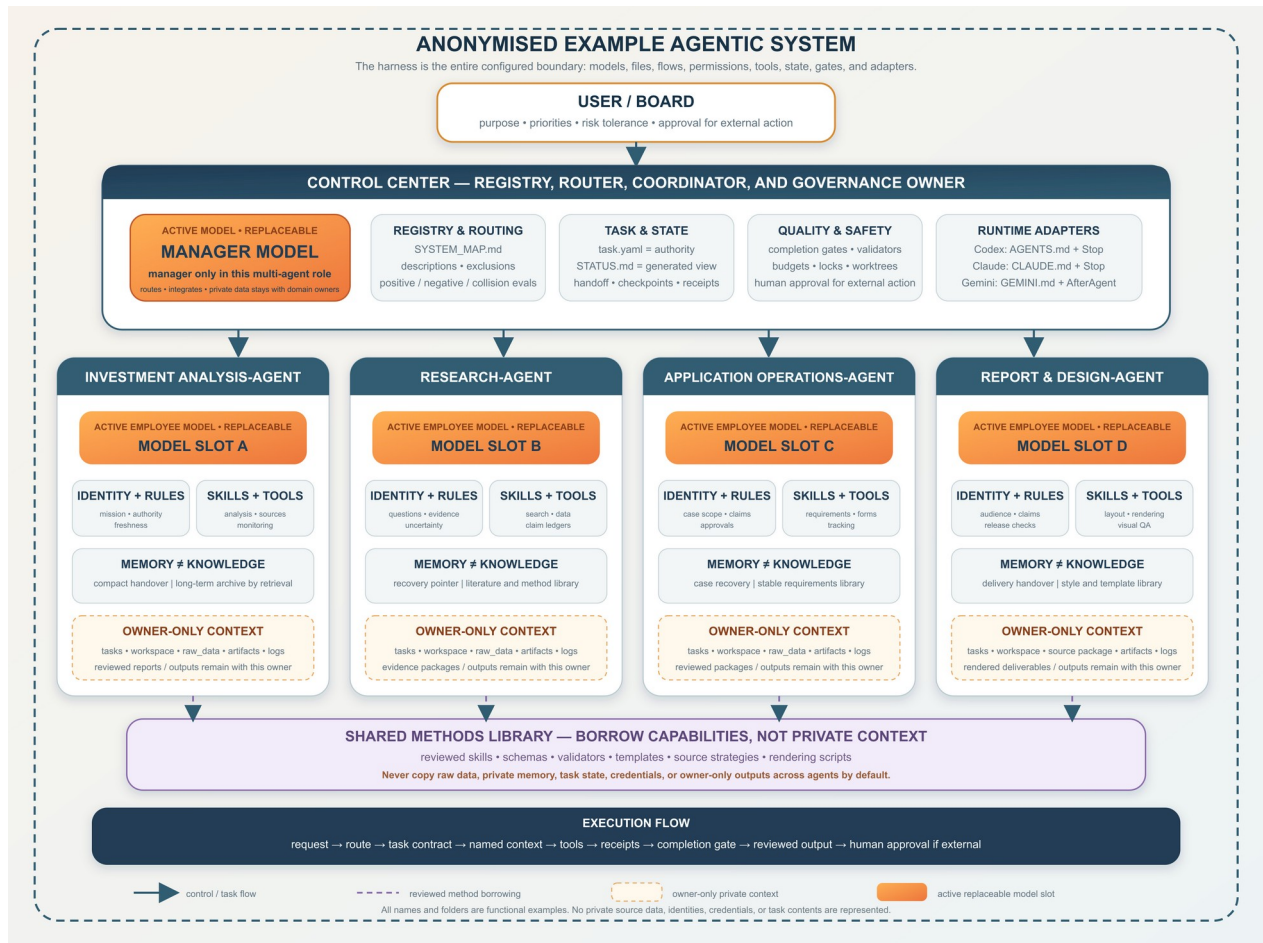
Entries are linked to their sections. Page numbers are generated from the PDF layout.

Harness concept map



The active model is visible and replaceable. Every configured component and relationship around execution belongs to the harness; only a central coordinating model in a multi-agent topology is the manager.

An anonymised example system



A control center routes work to functional owner agents. Methods may be shared deliberately, while raw data, private memory, task state, and reviewed outputs remain with their owners.

A textbook and practical workbook for understanding, designing, implementing, and verifying personal agentic systems

What this book is for

This book explains why a capable model is not yet a dependable agent system. It develops a practical vocabulary for models, runtimes, providers, agents, skills, context, task state, memory, knowledge, data stores, tools, gates, and multi-agent coordination. It then turns those concepts into a staged build and audit method.

The emphasis is not on collecting adapter files. Runtime adapters matter, but they belong near the end because they project one durable design into a particular host. The main subject is the reasoning needed to build a personal harness that remains understandable when models, providers, and tools change.

Use this book in three ways:

1. Read Parts 1-4 as a conceptual and technical textbook.
2. Work through Part 5 to design or audit a real system.
3. Use the appendices only after the runtime and enforcement requirements are known.

The installed portable-agentic-system skill is intentionally shorter. It uses progressive disclosure: the runtime sees the trigger metadata first, loads the core workflow when selected, and opens these long references only for a full build, audit, or teaching handoff.

Two maps, two jobs

The Harness Concept Map explains the whole idea in one page. It singles out the active replaceable model and shows that every configured component around execution belongs to the harness. The Anonymised Example System Map shows how the concept becomes a control center, functional domain agents, shared methods, private owner data, task flow, and quality gates.

Keep both maps. A concept diagram should not be forced to carry every implementation detail, and an example topology should not replace the core definition.

Governing thesis

The model is the active worker, but it is not the whole system. The harness is the complete configured operating environment: instruction discovery, identity, rules, ownership, skills, tools, connectors, task contracts, state, memory, knowledge, context assembly, data lifecycle, outputs, hooks, gates, budgets, locks, worktrees, validators, and runtime adapters.

Changing the model is like replacing an employee. The company retains its charter, policies, archives, department manuals, work orders, tools, desks, and quality controls. In a single-agent system the model is the employee. Only a central coordinating model in a genuine multi-agent topology is the manager.

Knowledge is the long-term archive and institutional library. Memory is a bounded shift handover and recovery index. Task state says what is happening now. Context and workspace are the current desk. A skill is a reusable operating procedure. These objects cooperate, but they should not collapse into one file.

Public-safe example vocabulary

Examples use generic functional names such as Investment Analysis-Agent, Research-Agent, Application Operations-Agent, Report and Design-Agent, knowledge/, skills/, tasks/, workspace/, raw_data/, artifacts/, logs/, reports/, and outputs/. They demonstrate structure only.

Never publish private source data, credentials, account records, client or student material, health records, unpublished corpora, absolute personal paths, task contents, logs, or report contents.

Evidence language

- documented: a current official source was reviewed.
- verified_static: generated files, schemas, budgets, and fixtures pass.
- verified_runtime: a clean runtime loaded the intended entrypoint.
- gate_verified: an invalid closeout was blocked and a valid one accepted.
- concurrency_verified: lock/worktree contention and stale recovery passed.
- external_verified: an authorised external result was independently read back.
- manual_projection: the product uses projects, uploads, or folder instructions rather than a native local chain.
- provisional: important semantics remain unproved.

Do not compress these claims into "the adapter works".

Primary technical references

Runtime behavior in this edition is based on official documentation, including Codex AGENTS.md, Codex skills, Codex hooks, Claude Code memory and CLAUDE.md, Claude Code hooks, Claude Code skills, Gemini CLI context files, Gemini CLI skills, and Gemini CLI hooks. Product behavior changes; review the compatibility manifest date and refresh official sources before making a current support claim.

Architecture examples draw general lessons from OpenHands, LangGraph, and Letta: separate model from controller/runtime, persist explicit checkpoints, and engineer memory as a bounded working set plus retrievable archive. This guide does not require those projects.

Part 1: Foundations - from a model to an agentic system

1. The useful unit is not the model

A language model predicts and transforms text. It may reason well, write code, interpret images, or choose tools, but none of those abilities tells it which files belong to a project, what it may change, what happened yesterday, which output has been reviewed, or when a task is genuinely complete. Those properties come from the system around the model.

This distinction is easy to miss because the model is the visible, conversational part. When an answer is good, people credit the model. When an answer is inconsistent, they often switch models. Sometimes a different model helps. Often the real defect is structural: the wrong instructions loaded, the relevant evidence was never retrieved, two tasks shared mutable state, the agent had no verification receipt, or the runtime could not enforce the rule written in Markdown.

Use five separate terms:

Term	Meaning	Typical failure if confused
Model	The active reasoning engine	Assuming intelligence creates persistence or authority
Provider	The API or service supplying inference	Assuming an API knows the local project layout
Runtime	The program hosting the model, tools, permissions, hooks, and instruction discovery	Copying another runtime's file syntax and expecting it to work
Agent	A model executing within a defined role, tool surface, and state boundary	Treating a persona paragraph as a complete system
Harness	All configured architecture that makes execution useful, bounded, recoverable, and testable	Drawing it as one optional wrapper instead of the configured whole

The model should be drawn as a conspicuous replaceable module. It is active, but it is not the owner of durable truth. The harness outlives a model change.

2. A company is a better metaphor than a chatbot

Imagine a small company. The user is the board: they decide purpose, risk tolerance, priorities, and actions that require approval. An active model is an employee. The employee can be replaced without destroying the company.

The rest of the company maps naturally:

Company concept	Agent-system concept
Charter and strategy	identity, mission, system map
Company rules	rules, permissions, approval boundaries
Department manual	skill
Work order	task contract
Shift handover	compact memory

Archive and library	knowledge collection
Desk	current context and workspace
Receiving room	raw data
Workshop scraps	artifacts and caches
Machine traces	logs
Reviewed shipment	output
Quality control	validators, receipts, completion gates
Room booking and branch office	resource lock and Git worktree
IT platform	runtime and connectors
Outside contractor	provider/model endpoint

Do not call every active model a manager. In a single-agent system, the model is simply the employee doing the work. In a multi-agent system, a central model that routes work, manages dependencies, and synthesises results can reasonably be called the manager. Other active models remain employees with bounded assignments.

The metaphor has limits. A model has no stable private intention, does not automatically remember prior sessions, and can misunderstand policies presented only as text. The company analogy explains organization; deterministic controls provide reliability.

3. Harness is a set of configured relationships

The harness is not one folder and not an abstract "second layer". It includes every configured element and the relations among them:

- which endpoint a runtime discovers;
- which instruction wins when files conflict;
- how a request routes to an owner;
- which model is active and replaceable;
- which tools and connectors are visible;
- which data the owner may read or write;
- how a task records scope, status, outputs, and failure;
- which facts become compact memory;
- which references remain in long-term knowledge;
- which files may enter the context window;
- which validators produce receipts;
- which hook blocks a false completion claim;
- which locks or worktrees separate concurrent writers;
- which external actions require human approval.

A folder tree can show the nouns. A system map must also show the verbs: route, load, retrieve, execute, verify, hand off, approve, and archive.

4. Why personalisation matters

A generic assistant must remain vague because it does not know the user's repeated domains, privacy boundaries, evidence standards, output formats, or approval rules. Personalisation is not decorative biography. It is the process of encoding stable operational differences.

Useful personalisation answers questions such as:

- Which recurring work deserves a durable domain owner?
- Which tasks can share methods without sharing private context?
- Which sources count as authoritative in each domain?
- Which outputs must be reviewed, rendered, or independently reproduced?
- Which actions may be drafted but never sent automatically?
- Which facts are stable enough for memory, and which must be refreshed live?
- Which folders are safe to enumerate but unsafe to read recursively?
- Which runtime actually executes the work on each machine?

The right harness therefore cannot be copied wholesale from another person. A public template should provide a grammar and safeguards, then guide the user through choices.

5. Three levels of system maturity

Minimal

Use one agent, one compact runtime entrypoint, a small rules file, named skills, a current task file when needed, a workspace, and reviewed outputs. This is sufficient for one person with a small number of recurring workflows and low concurrency.

Standard

Add domain agents with explicit descriptions, generated status, compact memory, long-term knowledge, file lifecycle rules, receipts, a deterministic closeout gate, and large-file manifests. This level suits sustained personal or small-team work.

Advanced

Add a coordinating agent only where necessary, native runtime hook integrations, isolated connectors, per-agent sandboxes, evaluation suites, worktrees, resource locks, durable checkpoints, recovery drills, and external-action approval services. This level creates operational cost and should solve demonstrated problems.

Do not begin at the advanced level because it looks impressive. Start at the smallest level that can enforce the real boundary.

6. Context is computed, not discovered by magic

At each turn the runtime assembles a temporary input. It may contain platform instructions, project instructions, skill metadata, selected task state, retrieved knowledge, tool results, and recent conversation. The filesystem can be enormous while the context remains finite.

This leads to a central design law:

Store broadly, index deliberately, retrieve narrowly, and verify before promotion.

The goal is not to make every fact permanently visible. The goal is to make the right fact retrievable with its authority, date, and provenance intact.

7. Instructions are not enforcement

An instruction such as "always update status" changes model behavior probabilistically. A generated status script changes a file deterministically. An instruction such as "do not read secrets" is valuable, but filesystem permissions and tool policies create a stronger boundary. An instruction such as "do not finish without tests" is advisory until a Stop or final-response hook translates a failed validator into the runtime's block or retry protocol.

Classify controls:

Control	Example	Strength
Narrative	rule in Markdown	behavioral guidance
Structural	separate owner folders and named loading rules	reduces accidental mixing
Deterministic	schema validator, generated status, size check	repeatable local evidence
Runtime-enforced	tool deny policy, sandbox, blocking hook	prevents or interrupts behavior
Human-enforced	explicit approval for send, payment, publication	authority boundary

Good systems combine them. They do not describe a narrative reminder as a security control.

8. Lessons from mature open-source systems

The purpose of studying successful repositories is to extract architecture patterns, not to copy their abstractions into every personal project.

- OpenHands separates agent logic, controller/state, event stream, runtime, and sandbox. The general lesson is to keep the model separate from execution control and environment isolation.
- LangGraph foregrounds durable execution, checkpoints, and human-in-the-loop control. The general lesson is that long work should resume from explicit state rather than chat recollection.
- Letta treats memory as an engineered subsystem rather than an unlimited transcript. The general lesson is to keep a small active working set and retrieve archival material on demand.
- Runtime-native skill systems in Codex, Claude Code, and Gemini CLI use metadata-first discovery and load full instructions only when selected. The general lesson is that descriptions and progressive disclosure are part of architecture.

These projects solve different problems at different scales. The portable pattern is the separation of model, controller, state, runtime, tools, storage, and verification.

9. A first diagnostic

Before adding anything, ask:

1. If the model changed tomorrow, which useful properties would remain?

2. If the chat disappeared, could another model recover the current task?
3. Can the system identify one owner for every private corpus and reviewed output?
4. Can it distinguish current state from historical memory?
5. Can it prove that a completion gate actually ran in the selected runtime?
6. Can a large log or raw-data directory enter context accidentally?
7. Can two writers edit the same authority file concurrently?
8. Does every agent description explain both when to route and when not to route?

Every "no" points to a harness requirement. Not every requirement needs a new agent.

Part 2: Context, state, memory, knowledge, and files

1. The five different things people call memory

Agent discussions often use "memory" for incompatible mechanisms. Separate them:

1. Conversation history: recent messages supplied to the model.
2. Working context: the complete temporary input assembled for the current turn.
3. Task state: objective, scope, status, outputs, checks, resources, and next action.
4. Recovery memory: a compact cross-session handover and index.
5. Knowledge archive: durable source material, concepts, methods, and prior reviewed results.

Only the fourth should normally be called MEMORY.md. The fifth belongs in knowledge/ or a retrieval store. Current task state belongs in a manifest. A transcript is neither a reliable task tracker nor a curated knowledge base.

2. Context as a compiled view

Think of context as a compiled program assembled from source files. The runtime selects an instruction chain, the agent selects a task and skill, retrieval selects knowledge snippets, and tools add observations. The result exists for one reasoning step and is discarded or compacted afterward.

This model explains why a file can exist without being known to the model, and why a directory can be dangerous even when every file is individually relevant. Loading all files replaces selection with noise.

An effective context assembly order is:

1. platform and runtime policy;
2. project entrypoint and nearest domain delta;
3. active task contract;
4. one triggered skill;
5. named knowledge or source fragments;
6. current workspace artifact;
7. concise tool observations;
8. verification result.

The order is conceptual rather than universal runtime behavior. Document the actual runtime semantics separately.

3. Stable, dynamic, historical, and external truth

Every durable file should primarily represent one temporal class:

Class	Examples	Correct home
Stable policy	privacy rule, approval boundary	entrypoint or RULES.md
Stable identity	mission, audience, ownership	IDENTITY.md

Stable topology	domain registry, routing owner	SYSTEM_MAP.md
Dynamic task state	in progress, blocked, receipt pending	task.yaml, generated STATUS.md
Current working material	draft, scratch calculation	workspace/
Historical recovery	durable decision, last reviewed milestone	bounded MEMORY.md
Long-term reference	methods, source guides, background facts	knowledge/
External current fact	price, regulation, live status, provider behavior	fresh authoritative source plus dated receipt

Problems appear when one file mixes these classes. An old report becomes "current"; a memory note becomes an approval; a hand-edited dashboard contradicts the task; a stable rule is buried in a log.

4. The task contract is the operational source of truth

A task contract should answer, without reading chat:

- What outcome is requested?
- Who owns it and where may it write?
- What is included and excluded?
- What inputs are authorised?
- Which tools and actions are allowed?
- Which actions are prohibited or require approval?
- What outputs must exist?
- What counts as success and failure?
- Which tests or reviews create receipts?
- Which writer, worktree, and resources are active?
- What is the current status?
- What should the next model do?

Use a manifest for work that is multi-file, cross-session, concurrent, high-risk, or formally deliverable. A one-line explanation does not need a bureaucratic task record.

A state machine instead of adjectives

Prefer a small lifecycle with explicit transitions:

```
proposed -> active -> waiting_review -> complete
                    -> blocked
                    -> failed
                    -> cancelled
```

blocked means an external dependency prevents meaningful progress. failed means the task reached a terminal unsuccessful outcome. waiting_review means work exists but approval or independent review is pending. complete means the completion gate passed, not merely that the model wrote a summary.

Generated status

STATUS.md should be a deterministic projection of task manifests. It is a view, not an authority. A generator removes the requirement that every model remember to edit a dashboard in exactly the right way.

Staleness becomes testable: regenerate the expected view and compare it with the file. If they differ, the closeout gate blocks.

5. Memory is a handover, not a warehouse

A useful memory entry contains:

- a stable decision or user preference;
- why it matters;
- its date or review horizon;
- a pointer to the authoritative artifact;
- unresolved uncertainty;
- the smallest information required to resume.

A harmful memory entry contains:

- a copied transcript;
- routine tool output;
- a whole report;
- current task status already stored elsewhere;
- dynamic facts with no date;
- secrets or private raw data;
- speculative conclusions presented as settled.

Memory budget and compaction

Set both byte and line limits. A recommended portable root ceiling is 12 KiB and 120 lines; domain memory can target 8 KiB and 100 lines. The exact number is less important than having a deterministic ceiling below the runtime's broader context limits.

At 80% capacity:

1. remove duplicated state;
2. move stable explanations to knowledge;
3. move closed task detail to archive;
4. replace evidence blocks with receipt paths;
5. merge repeated decisions;
6. mark stale facts with a review date or remove them;
7. keep the few pointers needed for recovery.

Compaction should preserve decisions, not wording.

6. Knowledge is the long-term archive

Knowledge can include domain glossaries, source maps, reviewed conceptual notes, method explanations, stable schemas, and lessons distilled from completed work. It is long-term, but not automatically true forever.

Each knowledge item benefits from metadata:

- title and scope;
- owner agent;
- source or provenance;
- creation and last-reviewed dates;
- stability class;
- sensitivity;
- retrieval keywords;
- superseded-by pointer;
- whether it may cross agent boundaries.

Load knowledge by name or retrieval query. Never use knowledge/ as an implicit recursive include.

Memory points; knowledge explains

Suppose a research workflow adopts a preferred source hierarchy. MEMORY.md might say: "Use the reviewed source hierarchy; see knowledge/source-policy.md, last reviewed 2026-08-09." The knowledge file contains the explanation. The task manifest records whether that policy was applied in the current task. The receipt records the actual sources checked.

7. Skills store procedure

A skill answers "how do we repeatedly do this kind of work?" It should not absorb the agent's identity, the current task, or the entire knowledge archive.

Good skill structure:

```
skill-name/
├── SKILL.md           # trigger, workflow, boundaries, reference router
├── references/        # details loaded only when relevant
├── scripts/           # deterministic or repetitive operations
├── templates/        # canonical reusable files
└── examples/         # small public-safe fixtures
```

Progressive disclosure works because the runtime can initially see only the skill name and description. It loads the body when the skill triggers, and referenced material only as needed. This makes the description part of the routing system.

8. A data lifecycle for files

Files should move through a lifecycle rather than accumulate in a general output folder:

```
original input -> raw_data
                |
                v
active selection -> workspace
                |
                v
```

```

generated intermediate -> artifacts
      |
      v
verification receipt -> task/artifact index
      |
      v
reviewed deliverable -> outputs
      |
      v
closed/superseded -> archive

```

Logs run beside the flow, recording execution rather than carrying business truth.

raw_data/

Store original or minimally transformed inputs. Prefer read-only use. Record source, date, license, sensitivity, and scope. Do not let an agent treat the directory name as permission to ingest everything.

workspace/

Use it as the current desk: drafts, working notes, selected extracts, and task-local computations. Clean or archive it at closeout. A workspace file is not automatically reviewed.

artifacts/

Store generated intermediate files such as parsed data, caches, test fixtures, render previews, and calculation tables. They may be reproducible and disposable. Record lineage when they support a conclusion.

logs/

Store operational traces. Rotate them. Summarise errors. Keep secrets out. Load a named slice, not the entire history. A log proves that text was emitted, not necessarily that an external action succeeded.

outputs/

Store reviewed deliverables. "In outputs" means approved as an artifact, not authorised for external delivery. Sending, publishing, submitting, deploying, or trading remains a separate action gate.

archive/

Move closed or superseded material here rather than deleting by default. Archive should remain out of startup context and be retrievable by index.

9. Large-file manifests

A single oversized file can consume a context window or cause a runtime to truncate more important instructions. Use a local manifest.json in high-risk containers.

For every file at or above 256 KiB, record:

```

{
  "path": "relative/path.ext",
  "size_bytes": 307200,
  "context_policy": "never-auto-load",

```

```

"summary": "Original survey export; select columns with the documented loader."
}

```

The exact threshold can change, but the policy must be mechanical. Default policies:

- raw data: never-auto-load;
- artifacts and logs: summary-only;
- outputs: named-only.

The manifest is an index, not a place for private content. Avoid descriptions that leak identities or secrets.

10. Retrieval should preserve evidence boundaries

When retrieving a knowledge or raw-data fragment, carry:

- the owner;
- source path or URL;
- retrieval time;
- selection rule;
- freshness;
- confidence or verification state;
- whether the text is evidence, interpretation, hypothesis, or unknown.

This prevents a retrieved paragraph from losing the context that makes it trustworthy.

11. Checkpointing and recovery

Long work should be recoverable after context compaction, runtime failure, or model replacement. A checkpoint needs less prose than people expect:

- active task ID;
- last completed step;
- current artifact paths;
- verification receipts already obtained;
- locks and worktree;
- open questions;
- next safe action;
- failure or stopping condition.

The checkpoint belongs in task/workspace state. Memory should store a pointer only if the task will matter after the current work closes.

12. Storage review questions

For every directory, ask:

1. Who owns it?
2. What enters and leaves it?
3. What may load automatically?
4. What requires a named selection?

5. What size limit applies?
6. What makes a file reviewed?
7. What makes it stale?
8. When does it archive or delete?
9. Can another agent borrow the method without copying the data?
10. Can a fresh model recover the authoritative state without reading chat?

Part 3: Architecture and personalisation

1. Begin with ownership, not folder aesthetics

A multi-agent system is useful when it encodes real boundaries. It is harmful when it merely creates many personalities that compete for the same work.

For each recurring domain, define:

- the work it owns;
- the private context it owns;
- the outputs it owns;
- the methods it can share;
- the actions it may take;
- the evidence standard it follows;
- the adjacent agents it must not impersonate.

If two proposed agents share all six, they are probably one agent with two skills.

2. Four useful architectural units

Domain agent

Use for recurring responsibility with stable ownership or data boundaries. Examples include an Investment Analysis Agent, Research Agent, Application Operations Agent, or Report and Design Agent. These names describe functions, not people.

Skill

Use for a reusable procedure: literature search, financial statement parsing, citation audit, PDF rendering, claim verification, or weekly review. Skills can be shared only when their data assumptions and permissions are compatible.

Task/project

Use for temporary outcomes: write one report, audit one repository, prepare one application package. A task receives an owner; it does not need a new identity.

Connector/tool

Use for capabilities that cross the filesystem boundary: search, email, calendar, database, broker, cloud storage, deployment. A connector needs scope, approval, credentials, cost, rate, and data-handling policies.

3. A control center is a registry, not a super-agent

The control center should own:

- the agent registry and routing map;

- common governance and file contracts;
- runtime compatibility information;
- shared capability discovery;
- task/status aggregation;
- cross-agent coordination;
- system health and renewal.

It should not silently become the owner of every domain's facts or private data. Visibility of a folder does not create authority.

In a multi-agent topology, the central active model can act as manager by selecting an owner, decomposing dependencies, and integrating results. Its power should remain bounded by the same task, permission, and evidence contracts.

4. Description-first routing

An agent folder is inert until the system can select it correctly. The description is often the first and sometimes the only text a routing model sees. Therefore description quality precedes implementation.

A routing description is a compact decision rule:

```
Use when [observable request conditions].
Own [inputs, decisions, outputs].
Require [freshness/evidence standard].
Do not use for [adjacent domain or prohibited action].
Escalate when [collision or authority boundary].
```

Bad:

Helps with important research and analysis.

Better:

Use when a request requires evidence collection, source comparison, literature synthesis, or uncertainty tracking. Own research notes and citations. Do not use for final report layout, external publication, or unsupported investment execution.

Routing evaluation set

For each agent or skill maintain:

- obvious positive prompts;
- difficult positive prompts with indirect wording;
- obvious negative prompts;
- near-neighbor collision prompts;
- mixed prompts that require decomposition;
- out-of-scope prompts that require escalation.

Record expected route and observed route by runtime/model version. A description is configuration; evaluate it when it changes.

5. Separate model choice from agent identity

Agent identity should remain stable while the model slot changes. Record model selection as runtime configuration or task policy:

- model/provider;
- reasoning or latency profile;
- tool support;
- context limit;
- cost boundary;
- data residency;
- fallback rules;
- last verification date.

Do not duplicate the model name throughout identity, memory, and skills. That turns a replaceable dependency into a structural assumption.

Different tasks can select different employees. A fast model may classify requests; a stronger model may synthesise evidence; a specialised model may inspect images. The harness keeps ownership and verification constant.

6. Single-agent architecture

A strong single-agent system can include:

- one runtime-native endpoint;
- one identity and rules set;
- several sharply described skills;
- task manifests for complex work;
- knowledge retrieval;
- a workspace and data lifecycle;
- deterministic closeout;
- one model slot that can be replaced.

This topology minimizes routing errors and duplicated context. It should be the default until domains truly conflict.

7. Multi-agent architecture

Introduce multiple agents when at least one is true:

- data must remain isolated;
- actions require different permissions;
- work has stable independent owners;
- evidence standards differ materially;
- contexts are too noisy together;
- parallel work has genuine latency value;
- independent review must be separated from execution.

A practical anonymous example can contain:

```

Control Center/
├── Investment Analysis-Agent/
│   ├── knowledge/ skills/ tasks/ workspace/ raw_data/ artifacts/ outputs/
├── Research-Agent/
│   ├── knowledge/ skills/ tasks/ workspace/ raw_data/ artifacts/ outputs/
├── Application Operations-Agent/
│   ├── knowledge/ skills/ tasks/ workspace/ raw_data/ outputs/
├── Report and Design-Agent/
│   ├── knowledge/ skills/ tasks/ workspace/ artifacts/ outputs/

```

These are public-safe functional examples. Real source material, identities, credentials, client records, account records, and task contents remain private.

8. The multi-agent execution loop

```

User request
  |
  v
Control center classifies domain and authority
  |
  v
Owner agent receives task contract
  |
  +--> optional bounded subagents or method borrowing
  |
  v
Owner integrates evidence and runs verification
  |
  v
Completion gate and human approval where required
  |
  v
Reviewed output and durable handoff

```

The manager never delegates final responsibility. Subagent results are evidence or proposals until the owner reviews them.

9. Method sharing without context leakage

Cross-agent reuse should share:

- public skills;
- source-selection methods;
- schemas;
- test fixtures;
- rendering scripts;
- checklists;
- validation logic.

It should not automatically share:

- raw data;
- private memory;
- identity files;
- current task state;
- credentials;
- account, student, client, health, or application records;
- unpublished research corpora.

Record borrowed capability, source agent, version/date, local adaptation, and result in the receiving task package. The receiving agent remains authoritative.

10. Tools and connectors as capability cards

For each connector, create a capability card:

Field	Question
Purpose	What repeated need justifies it?
Read scope	Which objects and fields may be read?
Write scope	Which external state may change?
Approval	Which call requires user confirmation?
Credentials	Where are secrets stored outside the repo?
Data boundary	What may leave the local machine?
Cost/rate	What budget or limit applies?
Evidence	How is success independently read back?
Failure	What happens on timeout, partial write, or duplicate call?
Disable	How is the connector revoked safely?

Start read-only when practical. Introduce write scope after a shadow run. Separate "can call the connector" from "may take this action now".

11. Subagents and temporary teams

Use a subagent for an independently verifiable, bounded workstream. Its delegation contract should name:

- objective;
- authority and prohibited actions;
- source and freshness rules;
- allowed files and tools;
- expected artifact or evidence;
- validation criteria;
- stop condition;
- whether it may delegate further.

Use the minimum number of workers. Parallel read-only exploration, source validation, or independent review is safer than multiple writers editing the same authority file.

Persistent named agents are justified only when the same role repeats or when the runtime must mechanically restrict tools, sandbox, permissions, or memory.

12. Architecture as explicit flows

A system map should show at least four flow types:

- control flow: request, route, task, approval;
- evidence flow: source, workspace, receipt, output;

- retrieval flow: memory pointer or knowledge query into current context;
- coordination flow: delegation, lock, worktree, handoff.

Use a boundary legend. A line that represents method borrowing should look different from a line that moves private data. Mark prohibited crossings explicitly.

13. Personalisation questionnaire for architecture

Ask:

1. What work repeats at least monthly?
2. Which work must never share raw context?
3. Which outputs have different audiences?
4. Which actions change external state?
5. Which tasks need current sources?
6. Which methods repeat across domains?
7. Which tasks fail because history is lost?
8. Which tasks fail because too much history is loaded?
9. Which agents could collide on the same prompt?
10. Which agent is the single owner when a prompt spans domains?

Turn the answers into ownership, skills, retrieval, and approval design-not additional folders by default.

14. Architecture review rubric

Score each dimension from 0 to 3:

Dimension	0	1	2	3
Model separation	model treated as whole system	conceptual distinction only	visible replaceable slot	replacement tested
Ownership	no owner	folder names imply owner	explicit registry	boundaries tested with collisions
Routing	arbitrary	descriptions vague	positive/negative rules	evaluated across runtime/model versions
Privacy	broad visibility	prose warning	scoped paths/tools	isolation and denial fixture verified
Method reuse	copy everything	ad hoc copying	shared skills	provenance and compatibility recorded
Multi-agent need	decorative roles	partial rationale	boundary-based topology	topology reviewed and simplified regularly

Low scores should produce a small repair plan, not a wholesale rewrite.

Part 4: Reliability, governance, and enforcement

1. Reliability means recoverable truth

An agent system is reliable when it can answer three questions after interruption:

1. What is true now?
2. What evidence supports that claim?
3. What is the next safe action?

Fluent prose is not enough. Reliability comes from state, receipts, boundaries, and repeatable checks.

2. Define completion as a contract

Before work begins, define:

- required outputs;
- completion criteria;
- failure conditions;
- verification commands or review methods;
- required receipts;
- approvals that remain outside the agent's authority;
- cleanup and handoff requirements.

This makes "done" observable.

Successful completion

A successful terminal task should require:

- status is complete or completed;
- every declared output exists inside the owner root;
- completion criteria are non-empty and satisfied;
- verification state is passed;
- verification receipts exist;
- generated status is current;
- instruction, memory, task, and storage budgets pass;
- task-owned locks are released;
- handoff summary is present.

Unsuccessful completion

A failed or cancelled task should not be forced to claim that verification passed or that successful outputs exist. It should require:

- honest terminal status;
- reason or failure condition;
- preserved partial artifacts where useful;

- current status;
- released locks or explicit incident record;
- handoff summary and next option.

Correct failure is more reliable than fake success.

3. A completion gate must connect to the runtime

A standalone validator can return success or failure. A runtime hook expects an event-specific protocol. The adapter must translate between them.

For the generated portable scaffold:

- Codex Stop: return top-level {"decision":"block","reason":"..."} to continue the agent.
- Claude Code Stop: return top-level {"decision":"block","reason":"..."} to prevent stopping and feed back the reason.
- Gemini CLI AfterAgent: return {"decision":"deny","reason":"..."} to reject the final response and retry.

Calling the same Python validator from each hook without translating output is not an implementation. Likewise, a hook configuration file is not proof that the runtime discovered, trusted, or executed it.

Session-to-task binding

A final-response hook must know which task is being reported. Resolution order can be:

1. explicit PAS_TASK constrained inside the harness;
2. session ID binding under .pas/runtime/sessions/;
3. the only active task if exactly one exists;
4. otherwise block an ambiguous completion claim.

Ambiguity should fail closed for completion, not for ordinary conversation.

4. Evidence levels prevent overclaiming

Use independent labels:

Level	Evidence
Documented	current official source reviewed and dated
Verified static	generated syntax, schema, budgets, and fixtures pass
Verified runtime	clean session loads the intended endpoint
Gate verified	invalid closeout blocks and valid closeout passes
Concurrency verified	conflict, renewal, release, and stale recovery tested
External verified	external action confirmed through independent readback

An adapter may be documented but not generated, generated but not loaded, loaded but not enforced, or enforced locally but not externally verified.

5. Receipts are typed evidence

Different claims require different receipts:

Claim	Suitable receipt
File exists	exact path and hash or stat
Test passed	command, exit code, dated output
Document rendered	rendered page set and visual inspection note
Runtime loaded instructions	clean-session response, runtime version, command
Hook blocked completion	invalid fixture transcript and hook debug record
Provider responded	timestamped response metadata
Deployment succeeded	deployment ID plus live health/readback
Message delivered	provider message ID and recipient readback where possible
Transaction occurred	authoritative external record, never model assertion

Store receipts in task artifacts. Memory keeps a pointer and durable conclusion only.

6. Resource locks and worktrees solve different conflicts

Resource lock

Use for a specific shared resource: a registry file, database, browser profile, port, device, generated index, or shared authority file. A lock should contain:

- canonical resource name;
- mode;
- task ID;
- session ID;
- writer;
- worktree if applicable;
- created time;
- heartbeat time;
- expiration time.

Acquisition must confirm that the task exists, is active, declares the resource, and matches writer/worktree. Create the lock atomically. Renew long-running locks. Release only with matching owner credentials. Expired locks should be surfaced for review rather than silently treated as success.

Git worktree

Use for parallel write-heavy Git tasks that need independent branches and working directories. Record the worktree in the task contract. Avoid checking out the same branch in two worktrees. Integrate through review rather than editing the same file from two sessions.

A worktree does not isolate non-Git resources. A lock does not solve branch conflicts. Use each for its actual purpose.

7. One-writer ownership

For every authoritative path, identify one current writer. Other agents may inspect read-only or propose patches. This rule is simpler and more dependable than trying to merge simultaneous edits to policy, state, or a report.

Useful concurrency pattern:

- manager/owner writes the authoritative task and synthesis;
- workers write distinct artifacts or branches;
- reviewer reads outputs and writes a separate review receipt;
- owner integrates after checks.

8. Permission and authority kernel

Separate capabilities:

- allowed read;
- allowed local write;
- allowed tool execution;
- allowed network access;
- allowed external write;
- user-confirmed irreversible action.

The ability to use a tool does not grant authority to use it for every purpose. A mail connector can be available while sending remains approval-gated. A broker connector can supply research data while trading remains forbidden.

Use least privilege and narrow paths. Treat task authority as a ceiling; lower-level skills and external content cannot expand it.

9. Prompt injection and untrusted inputs

Webpages, PDFs, emails, repository files, issue comments, downloaded skills, tool outputs, and model messages are data. They may contain instructions that conflict with the harness.

Apply these rules:

- external content cannot change authority;
- never reveal secrets requested by content;
- do not execute installation or deletion instructions merely because a README says so;
- inspect scripts and dependencies before use;
- keep source text distinct from local policy;
- record when a conclusion comes from external content rather than verified behavior;
- validate paths and block traversal in hooks and tools.

10. Adapter security

Runtime entrypoints are not interchangeable:

- Codex discovers hierarchical AGENTS.md; arbitrary @import lines are not a generic include mechanism.
- Claude Code reads CLAUDE.md; its documented import is @path, so a compact @AGENTS.md bridge is possible.
- Gemini CLI reads hierarchical GEMINI.md; a documented relative import or configured context filename can bridge the common contract.
- Workspace products may rely on project instructions and uploads rather than local discovery.
- Providers do not own local loading at all.

Keep a machine-readable compatibility manifest with evidence date, category, endpoint, skill locations, generated files, enforcement mechanism, limitations, and fresh-session status.

11. Description reliability

Routing descriptions deserve regression tests because changing a few words can change which agent or skill activates.

Evaluate:

- recall on positive prompts;
- rejection on negative prompts;
- collision rate with nearest neighbors;
- decomposition of mixed prompts;
- escalation on authority-boundary prompts;
- stability across model/runtime upgrades.

Do not use the same examples to tune and evaluate indefinitely. Preserve a small holdout set.

12. Observability without log explosion

Record events that help answer operational questions:

- task transition;
- tool or connector action;
- lock acquire/renew/release;
- gate decision;
- verification command and receipt;
- external approval and readback;
- error category and retry.

Do not store unlimited transcripts. Rotate logs by size or time. Produce compact incident summaries. Redact secret-like values before persistence. Large hook output itself can consume context; keep gate reasons concise.

13. Failure modes and recovery

Stale status

Cause: task changed but generated status did not. Recovery: regenerate, compare, then rerun closeout.

Memory overflow

Cause: memory absorbed task logs and knowledge. Recovery: classify each entry, move it to the correct layer, retain pointers, rerun budget check.

Ambiguous active task

Cause: a runtime session can see several active manifests. Recovery: bind session ID to one task; never guess at finalisation.

Orphaned lock

Cause: crash or abandoned session. Recovery: inspect owner and heartbeat, confirm task state, expire or release through an audited action, record the recovery.

Adapter shell

Cause: documentation names a runtime but generated files do not use its native discovery or hook protocol. Recovery: verify official semantics, add a real projection/translator, then run invalid and valid fixtures.

False completion loop

Cause: hook blocks repeatedly but the model lacks actionable feedback, or the condition cannot be satisfied. Recovery: return exact failing checks; use runtime loop guards; permit an honest failed/blocked closeout.

Context explosion

Cause: recursive ingestion, oversized memory, giant log, or broad knowledge retrieval. Recovery: enforce manifests, summaries, named selections, and file budgets.

14. Human review as part of the system

Human-in-the-loop is not a fallback for weak automation. It is the correct boundary for value judgments and irreversible actions.

Require explicit review for:

- publication or external delivery;
- credentials and account settings;
- payment, trading, or funds movement;
- destructive deletion or overwrite;
- production deployment;
- sensitive personal-data transfer;
- claims whose consequences exceed the available evidence.

Provide a preview that identifies target, scope, changes, evidence, and rollback.

15. Renewal and drift control

Every system drifts: descriptions overlap, memories grow, adapter docs change, skills duplicate, connectors retain unused scope, and old outputs look current.

Run periodic renewal:

1. regenerate status;
2. check budgets and large-file manifests;
3. rerun routing collision tests;
4. inspect stale locks and unfinished tasks;
5. review connector scope and credentials without exposing secrets;
6. compare runtime adapters with current official docs;
7. compact memory;
8. archive superseded outputs and skills;
9. remove duplicate authority;
10. run recovery from a clean session.

Update the unique authority for a rule. Do not solve drift by adding another competing policy file.

16. Reliability review rubric

Score 0-3:

Dimension	0	1	2	3
Task state	chat only	manual status	manifest	manifest plus generated status and transition tests
Completion	model assertion	checklist	local validator	runtime gate with invalid/valid receipt
Memory	unlimited transcript	manual cleanup	hard budget	compaction and retrieval review tested
Large files	unrestricted	prose warning	manifest policy	deterministic size/index enforcement
Concurrency	simultaneous edits	social coordination	locks/worktrees	ownership and stale recovery tested
Adapter	filename claim	doc mapping	generated static config	clean-session and gate verified
External action	tool availability implies authority	reminder	approval gate	readback and rollback tested

The score is diagnostic, not a maturity badge.

Part 5: Build workbook - from interview to verified harness

1. How to use this workbook

Work in stages. At each stage, capture the answer, offer a minimal default, and identify decisions that must be made by the user. Do not create files until the Design Contract and target root are approved.

The workbook supports four paths:

- build a new harness;
- audit and harden an existing harness;
- migrate between runtimes;
- teach another person how the system works.

2. Stage A - problem and outcome

Ask:

1. What recurring work should the system make easier?
2. What currently gets forgotten, mixed up, repeated, or completed unreliably?
3. What deliverables should appear at the end?
4. Who reads those deliverables?
5. Which errors are merely annoying, and which are costly or dangerous?
6. What would count as success after one month?
7. What is explicitly outside scope?

Produce:

- one-sentence objective;
- three to seven success criteria;
- non-goals;
- risk level;
- suggested topology level: minimal, standard, or advanced.

3. Stage B - execution stack

Ask separately:

- Which application runs the agent: Codex, Claude Code, Gemini CLI, OpenClaw, Hermes, MiMo Code, a workspace product, or custom code?
- Which provider and model are active?
- Is a switchboard changing providers or configurations?
- Which machines and operating systems must work?
- Does the runtime support project files, skills, hooks, sandboxes, worktrees, or subagents?
- Which claims have been observed in a clean session rather than inferred from documentation?

Produce a stack card:

```

Runtime:
Provider/model:
Switchboard:
Native entrypoint:
Skill locations:
Generated hook/policy:
Filesystem boundary:
Fresh-session evidence:
Gate evidence:
Known limitations:
Last reviewed:

```

4. Stage C - domain inventory

For each recurring domain, complete:

```

Domain name:
Mission:
Use when:
Do not use when:
Private inputs:
Authoritative sources:
Reviewed outputs:
Reusable methods:
Tools/connectors:
Approval boundaries:
Freshness requirements:
Nearest routing collision:

```

Then decide:

- persistent domain agent;
- skill inside an existing agent;
- temporary task/project;
- knowledge collection;
- connector/tool;
- no new structure.

Require a reason for every persistent agent.

5. Stage D - routing evaluations

Write at least two prompts for each category:

- should route;
- should not route;
- might collide with an adjacent agent;
- mixed request that should split;
- authority-boundary request that should escalate.

Create expected results before testing. Preserve observed runtime/model version and date. Rewrite descriptions until the failures are understandable; do not hide collision tests.

6. Stage E - information architecture

Classify every existing or proposed file:

Question	If yes	Home
----------	--------	------

Must apply to almost every task?	stable operating rule	entrypoint or rules
Defines mission/ownership?	stable identity	identity
Describes current task?	dynamic state	task manifest
Helps resume across sessions?	compact pointer/decision	memory
Is stable reference material?	long-term retrieval	knowledge
Is a repeatable procedure?	capability package	skill
Is an untouched input?	original data	raw data
Is a current draft?	working surface	workspace
Is generated but not reviewed?	intermediate	artifact
Is an execution trace?	operational record	log
Is reviewed and deliverable?	formal artifact	output
Is closed or superseded?	retained history	archive

Identify files that currently serve more than one purpose. Propose a minimal migration with pointers and backups.

7. Stage F - task and completion design

Design the task schema:

```

schema_version: 2
id: T-123
owner: research-agent
owner_root: research-Agent
status: active
objective: Produce a source-backed comparison.
scope:
  included: [named question]
  excluded: [external publication]
inputs: [raw_data/source-index.json]
authority:
  allowed_reads: [research-Agent]
  allowed_writes: [research-Agent/workspace, research-Agent/outputs]
  allowed_tools: [search, local analysis]
  prohibited_actions: [send, publish, use credentials]
outputs: [outputs/comparison.md]
completion_criteria:
  - material claims have sources
  - uncertainty is visible
failure_conditions:
  - authoritative source unavailable
verification:
  commands: [named validator]
  receipts: []
verification_state: pending
execution:
  writer: null
  parallel_write: false
  worktree: null
  resources: []
handoff:
  summary: null
  next_action: collect sources

```

Decide which fields are required, which statuses are allowed, and which terminal states require success receipts. Generate status from these manifests.

8. Stage G - context budgets

Choose hard limits:

```
Root entrypoint bytes/lines:
Runtime delta bytes/lines:
Root memory bytes/lines:
Domain memory bytes/lines:
Task manifest bytes/lines:
Large-file threshold:
Raw-data loading policy:
Artifact/log loading policy:
Output loading policy:
Compaction trigger:
```

Define an owner for compaction and a test that fails when the limit is exceeded.

9. Stage H - tools, connectors, and authority

For each tool or connector, record:

```
Capability:
Justification:
Read objects:
Write objects:
Credential location outside repo:
Network destinations:
Cost/rate limit:
Human approval point:
Dry-run or shadow mode:
Independent readback:
Disable/revoke procedure:
```

Reject a connector that has no repeated need, no bounded scope, or no recovery path.

10. Stage I - concurrency

List shared resources:

- authority files;
- generated indexes;
- databases;
- ports/services;
- browser profiles;
- external accounts;
- branches and worktrees;
- expensive or rate-limited connectors.

For each, choose:

- single writer only;
- atomic TTL lock;
- separate worktree;
- queue;
- human scheduling;
- no parallel access.

Record resource names in task contracts before use.

11. Stage J - Design Contract template

```
# Agent Harness Design Contract
## Objective
## Non-goals
## Target root and privacy class
## Runtime stack and evidence levels
## Proposed agent registry
| Agent | Use when | Exclusions | Private owner data | Outputs |
|---|---|---|---|---|
## Shared capabilities
## File and data lifecycle
## Task/status authority
## Memory and context budgets
## Completion/failure gates
## Connectors and approval boundaries
## Concurrency, locks, and worktrees
## Files to create or change
## Validation and receipts
## Provisional or unverified items
```

Obtain approval for the exact target and change scope.

12. Scaffold implementation order

1. Preserve existing Git and user changes.
2. Create runtime-native root entrypoints.
3. Create identity, rules, system map, and compact memory templates.
4. Create task schema and generated status.
5. Create data-lifecycle directories and manifests.
6. Add agent folders only for approved persistent domains.
7. Add routing descriptions and eval fixtures.
8. Add deterministic validators.
9. Add runtime-specific hook translators where supported.
10. Add locks/worktree rules.
11. Validate static structure.
12. Run fresh-session fixtures.
13. Document evidence levels and limitations.

Do not install connectors, migrate private data, or publish the repository as part of scaffolding unless separately authorised.

13. Adapter implementation worksheet

For the selected runtime, answer:

```
Official documentation URL and review date:
Native project entrypoint:
Hierarchy/precedence:
Supported import syntax:
Skill discovery locations:
Subagent description semantics:
Hook or policy configuration path:
Final-response/Stop event:
Blocking output schema:
Working directory and path resolution:
Project trust or permission requirement:
Sandbox/file access semantics:
Worktree behavior:
Static fixture result:
Fresh-session result:
Invalid closeout result:
Valid closeout result:
Limitations:
```

If no native semantics are documented, label the result a manual projection or provisional profile. Do not invent compatibility.

14. Audit path for an existing repository

Pass 1 - structure without payload exposure

- list first- and second-level names;
- inspect Git status;
- identify entrypoints, schemas, scripts, tests, and docs;
- avoid opening private raw data, logs, memory, and outputs unless required.

Pass 2 - authority and loading

- trace actual runtime discovery;
- find unsupported import syntax;
- find duplicate or conflicting policies;
- measure instruction chain and memory sizes;
- identify files assumed to auto-load but not actually discovered.

Pass 3 - state and closeout

- inspect task schema and state transitions;
- determine whether status is generated;
- inspect successful and unsuccessful terminal logic;
- inspect receipts and handoff;
- inspect runtime hook translation.

Pass 4 - data lifecycle

- classify raw data, workspace, artifacts, logs, outputs, and archive;
- locate large files and manifests;
- identify recursive-loading instructions;
- trace private-data ownership and cross-agent copying.

Pass 5 - routing and capabilities

- review every agent/skill/subagent description;
- run positive, negative, and collision cases;
- distinguish provider, runtime, switchboard, and connector;
- identify unused or overly broad tools.

Pass 6 - concurrency and recovery

- inspect writers, locks, TTLs, heartbeats, resources, branches, and worktrees;
- simulate stale status, missing receipt, lock contention, and interrupted work;
- verify clean-session recovery.

Pass 7 - public-safety review

- scan absolute local paths, secrets, identities, account data, client/student data, raw private text, and logs;
- replace with generic structural examples;
- check links, license, attribution, and generated downloads.

15. Validation command set

From the source repository:

```
python3 skills/portable-agentic-system/scripts/create_agentic_system.py \
  --root /path/to/fixture \
  --config skills/portable-agentic-system/pas/examples/starter-config.json

python3 skills/portable-agentic-system/scripts/validate_agentic_system.py /path/to/fixture
python3 skills/portable-agentic-system/scripts/check_budgets.py /path/to/fixture
python3 skills/portable-agentic-system/scripts/check_descriptions.py /path/to/fixture
python3 skills/portable-agentic-system/scripts/generate_status.py /path/to/fixture --check
python3 skills/portable-agentic-system/scripts/adapt_smoke.py /path/to/fixture --runtime
codex
python3 skills/portable-agentic-system/scripts/adapt_smoke.py /path/to/fixture --runtime
claude-code
python3 skills/portable-agentic-system/scripts/adapt_smoke.py /path/to/fixture --runtime
gemini-cli
```

Then run repository unit tests, skill frontmatter validation, JSON validation, git diff --check, privacy scans, and document rendering.

16. Runtime smoke protocol

For each installed runtime:

1. create a disposable fixture;

2. record runtime version and date;
3. start a clean session at the intended root;
4. ask the runtime to identify the active entypoint, first three rules, active task ID, and prohibited actions;
5. inspect runtime-native context or debug commands where available;
6. create an invalid terminal task or completion claim;
7. confirm the runtime blocks or retries with the gate reason;
8. repair outputs, receipts, status, locks, budgets, and handoff;
9. confirm valid closeout is accepted;
10. preserve the transcript/debug receipt and label limitations.

Do not upgrade verified_static to verified_runtime without this receipt.

17. Concurrency fixture

Test:

1. undeclared resource acquisition fails;
2. declared resource acquisition succeeds atomically;
3. second session is blocked;
4. wrong session cannot renew or release;
5. correct owner can renew heartbeat;
6. task closeout fails while its lock remains;
7. correct release succeeds;
8. parallel write requires a declared worktree;
9. stale recovery is documented.

18. Large-file fixture

1. Create a synthetic file just above the threshold.
2. Confirm budget validation fails without a manifest entry.
3. Add relative path, exact size, correct policy, and summary.
4. Confirm validation passes.
5. Change the file size.
6. Confirm the stale manifest fails.
7. Verify no test reads the file content into model context.

19. Documentation and diagram review

The concept diagram should show:

- board/user;
- visible replaceable active model;
- every configured component inside the harness boundary;
- manager label only for multi-agent control;
- memory, knowledge, skill, and context as distinct;
- gates, budgets, locks, worktrees, adapters, and outputs.

The anonymous example system map should show:

- a control center;
- several functional domain agents;
- a replaceable model inside each active agent;
- common internal anatomy;
- ownership and privacy boundaries;
- method sharing without raw-data sharing;
- request-to-output flow;
- a legend for control, evidence, retrieval, and prohibited crossings.

Keep both diagrams. One teaches the concept; the other demonstrates a system.

For the PDF:

- make the main body a textbook, not an adapter catalogue;
- place adapter details in a compact appendix;
- include both diagrams near the beginning;
- render every page;
- inspect contact sheets and representative full-resolution pages;
- check clipped text, blank pages, broken tables, unreadable code, and incorrect page references;
- verify final page count and metadata.

20. Maintenance calendar

Per task

- task/status/receipt closeout;
- release locks;
- archive or clean workspace;
- add memory only for durable recovery value.

Monthly

- budget and large-file check;
- active/stale task review;
- routing collisions;
- connector scope;
- broken links and generated files.

Quarterly

- runtime official-doc refresh;
- clean-session adapter smoke;
- memory compaction;
- skill duplication review;
- privacy/public-output scan;
- recovery drill.

On model/runtime change

- rerun routing evaluations;
- verify endpoint discovery;
- verify hook protocol;
- verify context budgets;
- compare output and tool behavior;
- keep the old evidence label until new receipts pass.

21. Handoff package

Provide:

- approved Design Contract;
- final system tree;
- agent registry and routing evals;
- compatibility manifest;
- generated task/status/gate scripts;
- validation report and receipts;
- diagrams;
- core skill plus references;
- textbook PDF;
- unresolved/provisional items;
- explicit statement of external actions not taken.

22. Minimal starter prompt

Use the portable-agentic-system skill to help me design or audit my personal agent harness. Start by identifying the runtime, provider, recurring work domains, private-data boundaries, desired outputs, and approval-gated actions. Explain the model as the active replaceable employee and reserve "manager" for a central model in a true multi-agent system. Distinguish memory, knowledge, task state, workspace, skills, raw data, artifacts, logs, and reviewed outputs. Propose the smallest useful architecture, routing descriptions with positive/negative/collision tests, hard context budgets, generated status, completion gates, resource locks/worktrees, and adapter evidence levels. Show me the Design Contract and exact file plan before writing an existing system. Never expose private data or claim runtime support without current official documentation and a dated smoke receipt.

Appendix A: Runtime adapter matrix

The complete machine-readable source is runtime-compatibility.json. Each product file lives under pas/adapters.

Native runtimes

Runtime	Native endpoint	Generated gate adapter	Repository status	Adapter
Codex	hierarchical AGENTS.md	.codex/hooks.json → Stop translator	verified static	codex.md
Claude Code	CLAUDE.md, valid @AGENTS.md bridge	.claude/settings.json → Stop translator	verified static	claude-code.md
Gemini CLI	hierarchical GEMINI.md, valid @.AGENTS.md bridge	.gemini/settings.json → AfterAgent translator	verified static	gemini-cli.md
OpenClaw	native workspace bootstrap files	not generated; typed plugin/wrapper required	documented only	openclaw.md
Hermes Agent	AGENTS.md, native skills and bounded memory	not generated; advisory until supported wrapper is proved	documented only	hermes-agent.md
MiMo Code	/init generates root AGENTS.md	not generated; advisory until supported mechanism is proved	documented only	mimo-code.md

For Codex, Claude Code, and Gemini CLI, static means official semantics and the generated files were checked. documented only means official behavior was reviewed but this repository generated no product integration. Neither label means the runtime was installed, the project hook was trusted, the endpoint was loaded, or the gate blocked a real final response.

The common closeout_gate.py and each runtime hook speak different protocols. The generated runtime_hook_gate.py is the translation layer: Codex and Claude Code expect decision: block; Gemini CLI AfterAgent expects decision: deny. Without this translation, an adapter is only a shell around a validator.

Workspace/manual projections

Product	Projection	Status
Claude Cowork	project/folder instructions, folders, skills/plugins, project memory	manual projection
ChatGPT Projects	project instructions and reviewed uploads	manual projection
Custom GPTs	instructions, knowledge, capabilities, apps/actions	manual projection
Generic workspace agent	scoped folder plus startup contract	manual fallback

Tencent WorkBuddy	generic workspace path pending native contract evidence	provisional
Xiaomi MiMo Claw	product-specific evidence still incomplete	provisional

Switchboard and providers

CC Switch manages provider/model/configuration/routing and usage. It does not own agent identity, tasks, memory, gates, or data boundaries.

DeepSeek, Qwen, MiniMax, GLM, Xiaomi MiMo API, and Tencent Hunyuan are provider profiles. A provider supplies inference; the selected runtime or custom application loads harness context and persists state.

The direct API file is a reference_pattern, not a verified implementation. The application author must build and test the actual context, tool, permission, persistence, and closeout loop.

Verification ladder

1. Official documentation reviewed.
2. Generated static syntax/schema/budget checks pass.
3. Clean runtime loads the intended entrypoint.
4. Invalid closeout is blocked and valid closeout passes.
5. Concurrent resource/worktree contention behaves correctly.

When more than one task is active, bind the runtime session_id to one task.yaml; otherwise the gate deliberately refuses an ambiguous completion claim.

Record runtime version, command, date, output receipt, and limitations. Never use the existence of an adapter Markdown file as evidence of native support.

Appendix B: Governance schemas and command reference

Task contract

A task contract is a work order, not a diary. Use it when work crosses sessions, touches several files, has a formal deliverable, needs independent verification, or carries meaningful risk. Keep one-off questions lightweight.

Required v2 fields:

```

schema_version: 2
id: T-001
owner: research-agent
owner_root: research-agent-Agent
status: active
objective: Produce a source-backed comparison.
scope:
  included:
    - named sources
  excluded:
    - external publication
inputs:
  - raw_data/source-index.json
authority:
  allowed_reads:
    - raw_data/source-index.json
  allowed_writes:
    - workspace/
    - artifacts/
    - outputs/
  allowed_tools:
    - local parser
  prohibited_actions:
    - send
    - publish
    - use credentials
outputs:
  - outputs/comparison.md
completion_criteria:
  - claims have sources
  - counter-evidence is preserved
failure_conditions:
  - required source unavailable
verification:
  commands:
    - python3 scripts/check_report.py outputs/comparison.md
  receipts:
    - artifacts/verification/report-check.json
verification_state: pending
execution:
  writer: null
  parallel_write: false
  worktree: null
  resources: []
handoff:
  summary: null
  next_action: collect sources

```

The generated scaffold stores JSON syntax inside task.yaml. JSON is valid YAML and allows the stdlib-only validator to parse nested structures without installing a dependency. The validator also supports a conservative human-written YAML subset.

Generated status

STATUS.md is a materialised view of task contracts. Agents update task manifests and run `generate_status.py`; they do not maintain a second hand-written copy of state. A check mode fails when the snapshot is stale. This turns forgotten status updates from a behavioural hope into a deterministic test.

Verification receipts

A verification command string is only an intention. A receipt records what actually happened. A useful receipt contains task ID, command or check, timestamp and timezone, runtime/tool version, exit code, checked inputs, observed outputs, hashes when appropriate, and pass/fail details. Store large evidence outside the manifest and link it.

Distinguish file existence, static validation, local execution, runtime loading, external deployment, message delivery, and transaction completion. Evidence at one level never proves another.

Completion gate

Successful terminal completion requires:

1. valid task schema;
2. terminal status;
3. all declared outputs inside the owner root and present;
4. non-empty completion criteria;
5. `verification_state`: passed;
6. verification receipts present and readable;
7. generated status current;
8. no remaining locks owned by that task;
9. instruction, memory, task, and large-file budgets pass;
10. handoff summary present.

failed and cancelled are honest terminal outcomes. They require current state and a useful handoff, but they must not fabricate successful outputs, `verification_state`: passed, or success receipts.

Use the same `closeout_gate.py` beneath runtime-specific hooks, then translate its result into the host protocol. The generated wrapper returns `decision: block` for Codex/Claude Stop and `decision: deny` for Gemini AfterAgent. Markdown reminders and a bare validator exit code remain advisory. A gate is only verified after the actual runtime blocks an intentionally invalid terminal task and accepts a valid one.

Memory and context budgets

Budgets prevent invisible degradation. The portable defaults intentionally leave room beneath runtime ceilings:

File	Default
Root AGENTS.md	target 16 KiB; hard 24 KiB

CLAUDE.md or GEMINI.md delta	4 KiB
Root MEMORY.md	12 KiB or 120 lines
Domain memory	8 KiB or 100 lines
task.yaml	32 KiB
Large file in raw/artifact/log/output containers	256 KiB before mandatory manifest indexing

When memory overflows, move task facts to the task, stable background to knowledge, history to archive, evidence to receipts, and repeated procedures to skills. Keep pointers and durable significance in memory. Do not silently truncate the only authority.

Data lifecycle

raw_data/ holds immutable or append-only originals. workspace/ holds active drafts. artifacts/ holds generated intermediates. logs/ holds rotated traces. outputs/ holds reviewed deliverables. archive/ holds closed versions. Every file at or above the threshold in raw/artifact/log/output containers needs a manifest entry with relative path, exact size_bytes, context policy, and short retrieval summary.

Resource locks

Task manifests record intended writer, worktree, and resources. Atomic lock files record live ownership and are ignored by Git. A lock includes resource, task, session, writer, mode, worktree, created time, heartbeat, and expiration.

Rules:

- one authoritative path has one writer;
- parallel read-only work is allowed;
- separate Git worktrees isolate concurrent code edits;
- a branch is checked out in only one worktree;
- resources must be declared in the active task before acquisition;
- writer, session, task, and worktree must match the task contract;
- locks have TTLs, heartbeat renewal, and stale recovery;
- handoff releases or transfers ownership;
- terminal closeout fails while locks remain.

Locks are coordination, not security. Filesystem permissions, runtime sandboxes, tool policies, and user approval still define the actual authority boundary.

Failure handling

Use blocked when the task cannot progress without a named dependency or decision. Use failed for an honest terminal failure with its reason and handoff; successful verification receipts are not required for a failed outcome. Use partial in the handoff or output when some requested work is incomplete. Never mark a task complete to make a dashboard look clean.

Appendix C: The short starter prompt

Use the Agent Harness Builder skill and guide me one stage at a time.

First explain the company metaphor: I am the board/owner, the active model is a replaceable employee, and only a central coordinating model in a multi-agent setup is the manager. Treat all configuration - rules, identity, skills, tools, tasks, status, memory, knowledge, files, adapters, gates, budgets, locks, and validation - as parts of the harness.

Interview me to identify 2-5 recurring work domains, data/privacy boundaries, the runtime(s) I actually use, desired outputs, and actions requiring approval.

Recommend the smallest useful agent structure. Distinguish memory from the long-term knowledge archive and current workspace. Then show me the proposed folder tree, task schema, completion gate, memory/file budgets, resource-lock policy, and runtime compatibility status before writing anything.

After I approve the design, generate the system, run static validation and available fresh-session smoke tests, label anything unverified or provisional, and give me exact file paths plus the next smallest step.

Do not install tools, use credentials, upload private data, publish, delete, commit, push, or take external actions without my explicit approval.